

Stock Price Movement Prediction

1. Problem statement and objectives

L'obiettivo di questo progetto è realizzare un sistema in grado di analizzare i valori di mercato e prevedere il prezzo futuro di alcuni titoli azionari. Questo tipo di problema rientra nel campo del time series forecasting, cioè la previsione di valori futuri partendo da una sequenza ordinata nel tempo di valori passati: in questo caso la sequenza è composta dai prezzi giornalieri dei titoli azionari.

Ci sono metodi statistici, come ARIMA, che sono costruiti appositamente per serie temporali, e che riescono a estrapolare dei pattern dal dominio dei dati di training.

Nel progetto viene usato un modello di supervised learning: esso apprende una relazione tra alcune feature costruite dai dati passati e i valori futuri da predire. In particolare, il modello utilizzato ha l'obiettivo di predire il prezzo medio futuro dell'azione basandosi sul valore medio del giorno corrente e su quello del giorno precedente. Inoltre, le predizioni del modello vengono usate all'interno di un sistema di backtest per verificare se, nel caso fosse stata adottata una strategia di investimento basata su tali predizioni, esse avrebbero portato a un guadagno di capitale.

2. Data analysis approach and findings

I dati iniziali vengono scaricati usando la libreria yfinance, e vengono salvati su un file separato per ogni ticker, in modo da facilitarne il caricamento in momenti successivi.

Le colonne presenti nei dati raw sono:

Date	Open	High	Low	Close	Volume
------	------	------	-----	-------	--------

- Date: data di quotazione;
- Open: prezzo di apertura;
- High: prezzo massimo della giornata;
- Low: prezzo minimo della giornata;
- Close: prezzo di chiusura;
- Volume: numero di azioni scambiate.

A partire da questi dati vengono costruite le seguenti due feature:

1. avg_current: è il prezzo medio giornaliero dell'azione, dato da $(high + low) / 2$.
2. avg_prev_1: è il prezzo medio del giorno precedente. Esso viene ottenuto tramite uno shift (in avanti) di un giorno della colonna avg_current.

Tramite avg_current e avg_prev_1, il modello deve predire i prezzi medi dei sette giorni lavorativi successivi. Questi vengono ottenuti e aggiunti al dataset facendo vari shift

(indietro): per il giorno successivo viene fatto uno shift di -1 giorno, per il settimo giorno successivo uno shift di -7 giorni.

Durante lo sviluppo sono state valutate anche feature aggiuntive, come statistiche calcolate su finestre temporali (min, max, mean, std) e il prezzo medio di un numero maggiore di giorni precedenti. Tuttavia, aumentare il numero di feature non va a migliorare le prestazioni del modello. Questo comportamento probabilmente avviene perché i prezzi di giorni vicini sono molto autocorrelati. Inoltre, aggiungere troppe feature può introdurre rumore o ridondanza, senza dare al modello un'informazione realmente nuova.

3. Machine learning methodology and results

Il modello di machine learning scelto per il progetto è Ridge Regression, un modello simile alla regressione lineare classica, ma con un termine di regolarizzazione che penalizza coefficienti troppo grandi e rende il modello meno sensibile al rumore presente nei dati. L'errore che il modello cerca di minimizzare è:

$$Loss = \|X\beta - Y\|_2^2 + \alpha \|\beta\|_2^2$$

È stato scelto questo modello perché è semplice e stabile, e il problema affrontato non richiede un modello molto complesso; infatti, il prezzo medio del giorno successivo è fortemente legato ai prezzi dei giorni precedenti.

Sono stati considerati anche altri approcci, ma hanno portato tutti a risultati peggiori rispetto al Ridge Regression. I seguenti valori sono relativi alle predizioni fatte sul giorno successivo:

Model	Mean Absolute Error	Root Mean Squared Error
Ridge Regression	2.873	4.172
Polynomial Regression (Degree 2)	3.028	4.424
Polynomial Regression (Degree 3)	32.646	61.402
Decision Tree Regression	60.1334	80.504

Si può osservare che, mentre per la regressione polinomiale di grado 2 i risultati sono leggermente peggiori rispetto al Ridge Regression, per gli altri due modelli i risultati sono pessimi.

Le metriche usate per valutare il modello sono:

- Mean Absolute Error (MAE): indica di quanti dollari, in media, la previsione si discosta dal valore reale;
- Root Mean Squared Error (RMSE): penalizza maggiormente gli errori più grandi, quindi, è utile per capire se il modello commette previsioni molto sbagliate in alcuni casi.

I risultati medi ottenuti con Ridge Regression sono:

	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7
--	-------	-------	-------	-------	-------	-------	-------

MAE	2.873	4.509	5.679	6.669	7.524	8.369	9.138
RMSE	4.172	6.304	7.819	9.101	10.204	11.265	12.246

Come prevedibile, l'errore aumenta andando avanti nel tempo. La previsione del giorno successivo è più semplice, perché il prezzo, in generale, è molto vicino al prezzo corrente. Al contrario, prevedere il prezzo medio tra sette giorni è più complicato perché nel frattempo possono esserci più variazioni di mercato non osservabili dal modello.

4. Technical implementation details

Modules

Il progetto è organizzato in moduli, ognuno con una precisa responsabilità.

La configurazione generale del progetto si trova nel file `config.py`. Qui sono definiti i ticker disponibili, il periodo considerato, le feature usate dal modello, i target da prevedere, la percentuale di testing set e i parametri del backtest. I vari file del progetto sono strutturati in modo tale che, un cambiamento nel file di configurazione, permette di applicare le modifiche globalmente. Per esempio:

- Se si vuole testare il modello con un numero maggiore di feature di ingresso, basta aggiornare la costante `PREVIOUS_DAYS`, che indica di quanti giorni precedenti vogliamo fornire i valori medi giornalieri delle azioni al modello.
- Se si vuole testare il modello con delle feature statistiche (min, max, mean, std), basta aggiornare la costante `WINDOW_SIZES`, inserendo la dimensione, o le dimensioni, delle finestre temporali

La fase di download dei dati, tramite la libreria `yfinance`, è gestita da `data_loader.py`: il modulo scarica i dati storici per un ticker e li salva in un file CSV, che viene letto in più occasioni durante l'uso dell'applicazione.

La fase di preprocessing è gestita dal modulo `data_processor.py`. Qui vengono aggiunte le feature basate sul giorno corrente e sul giorno precedente, e le colonne target per i sette giorni futuri. I valori mancanti che inevitabilmente si vengono a presentare (per come viene costruito il dataset), in particolare il primo e gli ultimi sette, vengono rimossi. Infine, il dataset processato viene salvato in un file CSV.

Training and evaluation

Il training e la valutazione del modello sono implementati nel file `ridge_regression.py`, ed avvengono nel seguente modo:

1. Carica il dataset processato del ticker specificato dal relativo file CSV;
2. Divide i dati in training e testing set (80% - 20%) mantenendo l'ordine temporale, cioè senza mescolare i dati;

3. Cerca il miglior valore dell'iperparametro alpha usando un validation set. In particolare, esso viene ottenuto dal training set in modo che la divisione totale del dataset sia 70% per il training set, 10% per il validation set e 20% per il testing set.
4. Standardizza le feature usando StandardScaler;
5. Addestra il modello di Ridge Regression;
6. Calcola le predizioni sul test set;
7. Valuta il modello con MAE e RMSE per ciascun target;
8. Salva il modello, lo scaler e i risultati su file

La scelta di non riordinare il dataset è fondamentale perché altrimenti si andrebbe a perdere la relazione che c'è tra i vari valori. Tutte le funzionalità del progetto si basano sul fatto che i dati sono in ordine cronologico: se questa condizione non è verificata, perdono di senso.

Backtest

Una parte importante del progetto è quella del backtest, implementato nel file `backtest.py`. In questa fase viene simulata una strategia di trading basata sulle predizioni del giorno successivo. Essa si basa sull'investire tutto il capitale se è previsto un rendimento maggiore di una certa soglia, altrimenti rimanere fuori dal mercato. Il rendimento atteso del giorno $t+1$ viene calcolato nel seguente modo:

$$EXPECTED\ RETURN_{t+1} = \frac{PREDICTED\ PRICE_{t+1} - ACTUAL\ PRICE_t}{ACTUAL\ PRICE_t}$$

Se il rendimento atteso è maggiore di una soglia pari a 0.001, cioè 0.1%, allora il capitale viene investito. In tal caso, viene ottenuto il rendimento reale dell'azione:

$$ACTUAL\ RETURN_{t+1} = \frac{ACTUAL\ PRICE_{t+1} - ACTUAL\ PRICE_t}{ACTUAL\ PRICE_t}$$

Inoltre, viene sottratto un costo di transazione pari a 0.1% ogni volta che il capitale viene investito o disinvestito.

Quindi, se nel complesso il modello funziona, con questa strategia dovremmo investire il capitale solo nei giorni in cui il rendimento è positivo (e maggiore della soglia).

Questa strategia viene poi confrontata con una strategia buy and hold, nella quale il capitale rimane sempre investito, cioè risente di tutti i rendimenti, sia positivi che negativi. È chiaro che, se il modello funziona, la strategia basata sul modello deve ottenere un capitale finale migliore di quello ottenuto dalla buy and hold.

5. Challenges encountered and solutions

La difficoltà principale incontrata è stata capire quali feature usare in input. Può sembrare naturale che aggiungere più feature con informazioni dei giorni precedenti,

non possa che migliorare i risultati. Tuttavia, dopo le prove effettuate, è emerso che l'aggiunta di più feature non migliorava davvero le prestazioni. La scelta che ho fatto è stata quella di usare solo due feature perché riduce la complessità del modello e limita il rischio di introdurre feature ridondanti o rumorose; inoltre, il training del modello viene fatto in tempo reale durante l'uso dell'applicazione, quindi non può richiedere un tempo troppo elevato.

Un'altra difficoltà è stata capire, durante la fase iniziale del progetto, come gestire correttamente la natura temporale e correlata dei dati. Nei dataset tradizionali spesso si divide il dataset in training e testing in modo casuale, per limitare il più possibile la formazione di pattern particolari tra i dati. In questo caso, invece, non sarebbe corretto perché il modello deve imparare da valori passati e prevedere valori futuri.

6. Eventual Future improvements

Analizzando le scelte progettuali fatte durante lo sviluppo del progetto, ho individuato alcuni possibili miglioramenti che potrebbero essere approfonditi in futuro.

Un primo aspetto riguarda le feature utilizzate con i vari modelli. Il confronto tra modelli (decision tree, polynomial regression e ridge regression) è stato fatto usando sempre due feature: il prezzo medio del giorno corrente e il prezzo medio del giorno precedente. Con queste feature, i modelli più complessi danno risultati pessimi, ma come cambiano i risultati di questi modelli se modifichiamo le feature? Si potrebbe quindi testare decision tree e polynomial regression con altre feature, che comprendono un set più ampio di dati sui giorni passati e qualche valore statistico che ne indica l'andamento generale.

Un secondo possibile miglioramento riguarda il confronto con modelli specifici per il time series forecasting. Nel progetto è stata utilizzata la Ridge Regression, ma esistono anche modelli progettati appositamente per lavorare con serie temporali, come ARIMA. Un confronto tra Ridge Regression e ARIMA sarebbe utile per capire quale approccio si adatta meglio al problema.